



Análise de complexidade de algoritmos



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Como analisar a complexidade de um algoritmo
- Experimento com caso real
- Análise assintótica
- Notações O , Θ , Ω
- Análises de algoritmos
 - ▣ pior, melhor e médio caso

Análise de algoritmos

- Como analisar a eficiência de um algoritmo?
 - ▣ Tempo de execução ou espaço ocupado em memória
- Possível solução: estimar o tempo exato de execução do algoritmo com base em alguns exemplos!
 - ▣ Vamos analisar um problema como exemplo de experimento...

Experimento

□ Problema:

- ▣ Dado um arquivo de texto contendo N inteiros gerados aleatoriamente
- ▣ Quantas triplas existem somando 0?

324110
-442472
626686
-157678
508681
123414
-77867
155091
129801
287381
....



324110
-442472
626686
-157678
508681
123414
-77867
155091
129801
287381
....

$$-442472 + 155091 + 287381 = 0$$

Experimento

```
int* leInteirosArquivo()
{
    FILE* file = fopen(FILENAME, "r");
    int* vetor = (int*)malloc(N * sizeof(int));
    int i = 0;
    int count = 0;

    while (!feof(file))
    {
        fscanf(file, "%d", &i);
        vetor[count] = i;
        count++;
    }
    fclose(file);
    return vetor;
}
```

Experimento

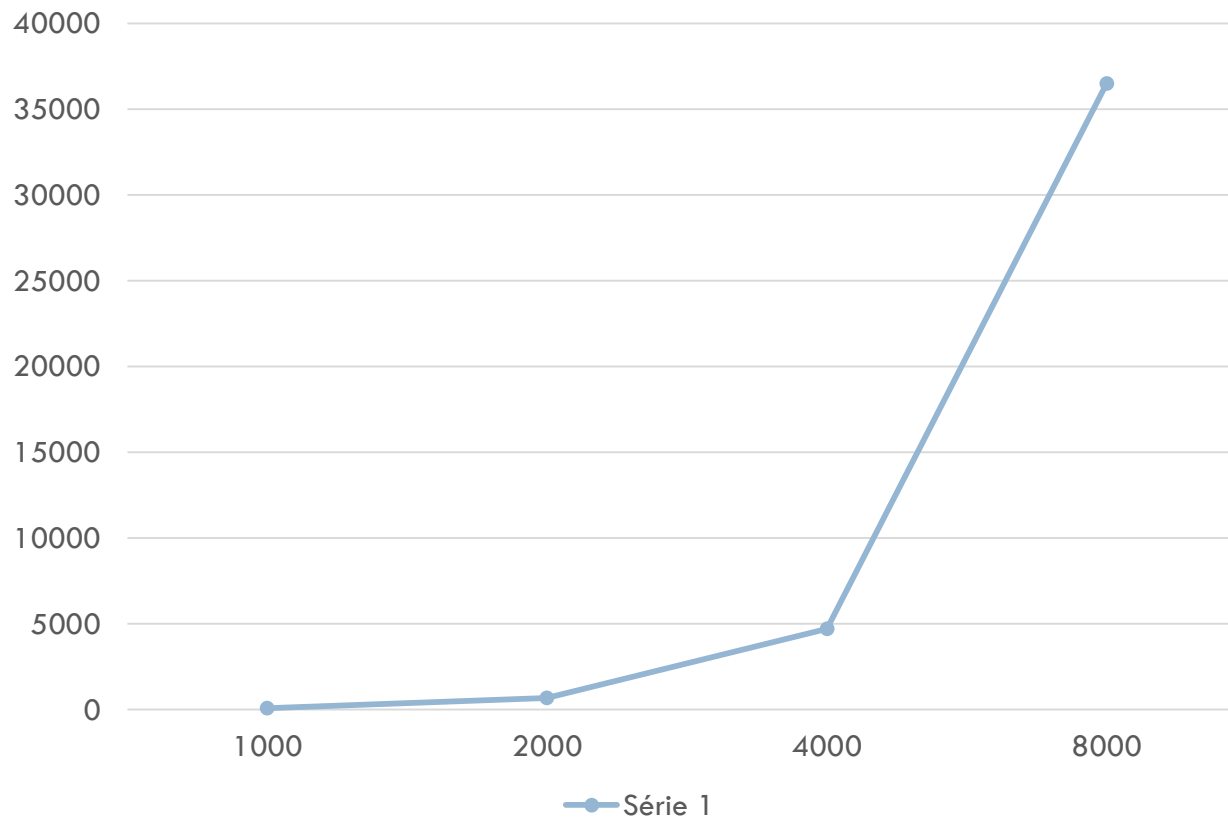
```
int main()
{
    int* vetor = leInteirosArquivo();
    clock_t ticks[2];
    ticks[0] = clock();
    int numTriplas = contaTriplas(vetor, N);
    ticks[1] = clock();
    double tempo = (ticks[1] - ticks[0]) * 1000.0 / CLOCKS_PER_SEC;

    printf("\nNumero de triplas que somam zero: %d", numTriplas);
    printf("\nTempo gasto: %g ms.", tempo);
    free(vetor);
    getchar();
    return 0;
}
```

Experimento

Exemplos:

tempo (ms)



Número de
inteiros

Arquivos: <https://github.com/aistrate/AlgorithmsSedgewick/tree/master/1-Fundamentals/1-4-AnalysisOfAlgorithms>

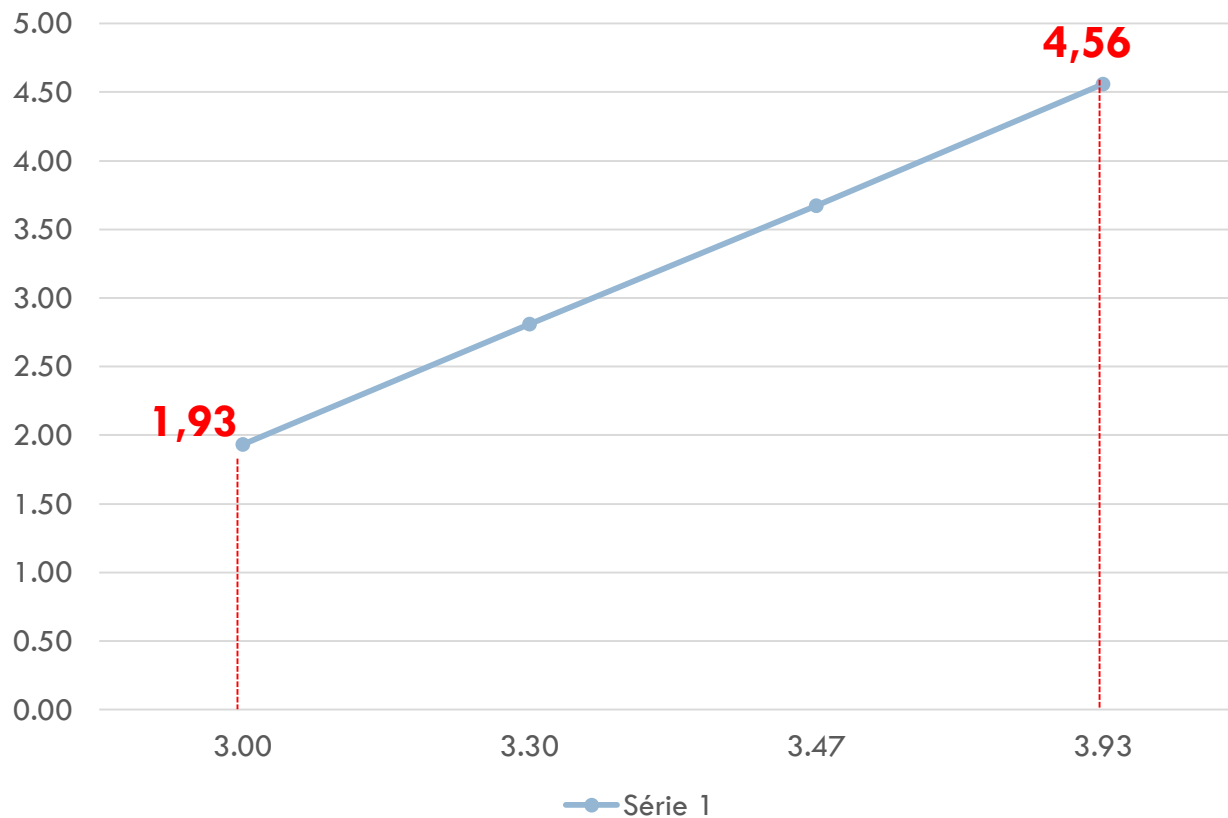
Experimento

- Previsão do tempo de processamento $T(N)$ para uma entrada com $N = 16000$ inteiros
 - ▣ Regressão linear simples
 - ▣ Conversão do gráfico para uma escala logarítmica (nos dois eixos)
 - *log-log plot*

Experimento

Exemplos:

$\log_{10}(\text{tempo (ms)})$



$\log_{10}(\text{Número de inteiros})$

Experimento

- $\begin{cases} 1,93 = a3 + b \\ 4,56 = a3,93 + b \end{cases}$
- $a \cong 2,82$
- $\log_{10}(T(N)) = 2,82 \cdot \log_{10}(N) + b$
- $T(N) = 10^{2,82 \cdot \log(N) + b} = 10^b N^{2,82} = cN^{2,82}$
- Estimando valor da constante c:
 - ▣ $T(8000) = 36500$
 - ▣ $T(N) = cN^{2,82}$
 - ▣ $36500 = c8000^{2,82} \Rightarrow c \cong 3,6 \cdot 10^{-7}$
 - ▣ $T(N) = 3,6 \cdot 10^{-7} N^{2,82}$

Experimento

- Algumas previsões:
 - ▣ $T(1\,6000)=4,29$ minutos
 - ▣ $T(32000)=30$ minutos
 - ▣ $T(64000)=213$ minutos = 3,5h
 - ▣ $T(128000)=25$ h
 - ▣ $T(10^6)=345$ dias

Análise de algoritmos

- Como analisar a eficiência de um algoritmo?
- Possível solução: estimar o tempo experimentalmente...
 - ▣ Problema: nem sempre isso é possível/conveniente
- É esperado que ocorram variações
 - ▣ Em função do hardware da máquina e uso do sistema operacional, principalmente
 - ▣ Tempo pode variar muito
 - Configurações da máquina
 - Uso do sistema operacional
 - Tamanho dos dados de entrada

Análise de algoritmos

- Tempo de execução depende de
 1. Custo de execução de cada instrução
 - Dependente da arquitetura
 2. Frequência de execução de cada instrução
 - Dependente do algoritmo
 - Foco se dá nos trechos de código mais frequentes

**Foco da análise
de algoritmos!**

Análise de algoritmos

- Nem sempre vale a pena todo o esforço para medir o tempo estimado de execução
- Mais importante do que o tempo exato (geralmente):
 - ▣ Ordem do crescimento do tempo de execução
 - “Como o tempo de execução de um algoritmo aumenta com o aumento do tamanho da entrada indefinidamente?”
 - Eficiência assintótica!
 - ▣ Em geral, um algoritmo que é assintoticamente mais eficiente será a melhor escolha
 - Exceto para as entradas muito pequenas

Eficiência assintótica de algoritmos

- Por que analisar a eficiência assintótica?
- Ordem de crescimento fornece uma caracterização simples da eficiência do algoritmo
 - ▣ Permite comparar de forma simples algoritmos alternativos
 - ▣ Nem sempre vale o esforço de estimar o tempo exato
- Podemos distinguir algoritmos que demoram segundos ou minutos daquele que executam por semanas, meses, ou até mais
 - ▣ em geral, mais importante do que comparar seus tempos exatos

Eficiência assintótica de algoritmos

- Estudo da **eficiência assintótica**
 - ▣ Análise de entradas grandes o suficiente para tornar relevante apenas a ordem de crescimento do tempo de execução
 - ▣ Se preocupa com o aumento do tempo de execução (ou memória) de um algoritmo a medida que o tamanho da entrada aumenta indefinidamente

Análise de algoritmos

- Exemplo: Considere os seguintes trechos do código abaixo.
- Quantas vezes cada um deles é executado?

```
int contaTriplas(int vetor[], int n)
{
    int contagem = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            for (int k = j + 1; k < n; k++)
                if (vetor[i] + vetor[j] + vetor[k] == 0)
                    contagem++;

    return contagem;
}
```

- `if (vetor[i] + vetor[j] + vetor[k] == 0)`
 - $N(N-1)(N-2)$ execuções
 - $N(N^2 - 2N - N + 2) = N^3 - 3N^2 + 2N$
- `int contagem = 0;`
 - Executado apenas uma vez. Irrelevante!

Análise de algoritmos

□ Exemplo:

□ $N^3 - 3N^2 + 2N$

▣ $-3N^2 + 2N$ considerado insignificante em relação a N^3 , para grandes entradas

▣ Ex: para $N = 1000$

▣ $-3N^2 + 2N = -2998000$

▣ $N^3 = 1.10^9$

▣ Para $N=1000$, $3N^2 + 2N$ equivale a 0,3% de N^3

▣ Logo, dizemos que a complexidade do algoritmo é:

▣ $T(N) = N^3$

▣ Note que no experimento realizado, estimamos

■ $T(N) = 3,6.10^{-7}N^{2,82}$

Notação assintótica

- Notações para descrever tempo de execução assintótica de um algoritmo
 - ▣ Descrito em termos de funções cujos domínios são o conjunto dos números naturais $N = \{0, 1, 2, \dots\}$
 - ▣ $T(n)$
 - Complexidade para uma entrada de tamanho n
 - Tamanho da entrada é normalmente definido por números inteiros

Notação assintótica

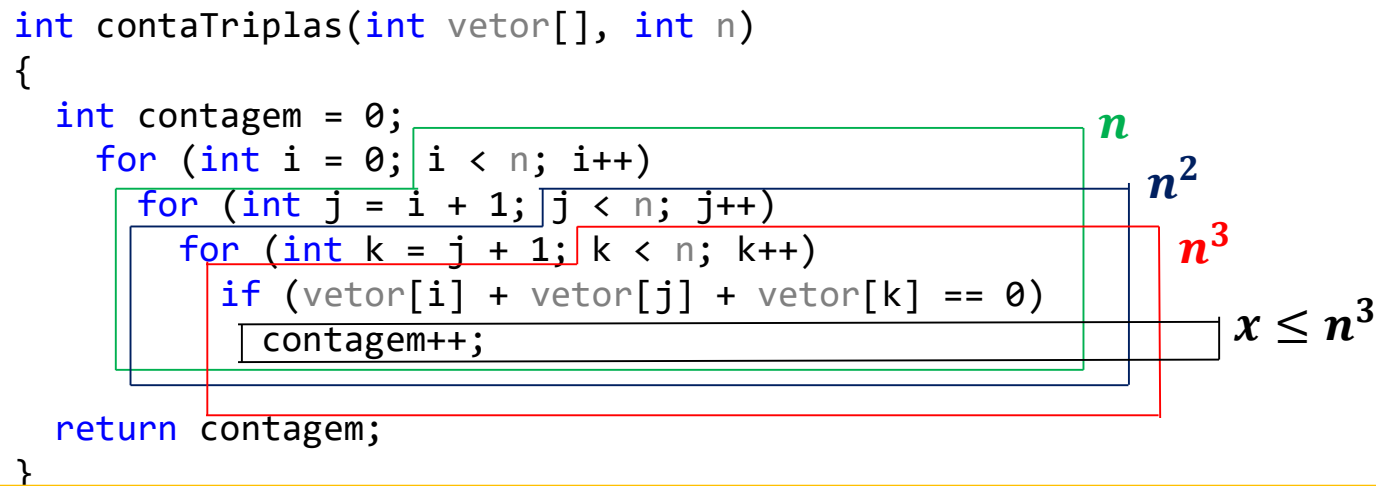
- Notações para descrever tempo de execução assintótica de um algoritmo
 - ▣ $T(n) = af(n) + g(n) + c$
 - Constantes (a e c) descartadas
 - $g(n)$ descartado caso $\lim_{n \rightarrow \infty} g(n)/f(n) = 0$
 - Ex: caso $g(n) = n^2$ e $f(n) = n^3$, n^2 é desprezado
 - Termos de menor ordem são descartados

Análise de algoritmos

□ Em outras palavras

- Apenas as instruções que são executadas mais frequentemente são levadas em consideração para analisar a complexidade do algoritmo

- Para $T(N) = N^3 - 3N^2 + 2N$, apenas N^3 foi considerado
- Como as instruções em vermelho possuem a maior frequência, dizemos que $T(n) = n^3$



The diagram shows a code block for a function `contaTriplas` with three nested loops. The loops are annotated with colored boxes and labels to show their complexity: the outer loop is green and labeled n , the middle loop is blue and labeled n^2 , and the inner loop is red and labeled n^3 . The innermost loop body, which includes an `if` statement and an increment, is enclosed in a red box. A label $x \leq n^3$ points to this innermost loop body. The entire code block is enclosed in a yellow box.

```
int contaTriplas(int vetor[], int n)
{
    int contagem = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            for (int k = j + 1; k < n; k++)
                if (vetor[i] + vetor[j] + vetor[k] == 0)
                    contagem++;
    return contagem;
}
```

Análise de algoritmos

- Mais um exemplo: cálculo do fatorial

```
int n = input;  
int fat = 1;  
for (int i = n; i > 0; i--)  
{  
    fat *= i;  
}
```

- $T(n) = 3 + n + 2n + 2n = 5n + 3$

- Simplificando:

- ▣ $T(n) = n$

Análise de algoritmos

□ Funções de custo comuns

T(N)	Nomenclatura
1	Constante
$\text{Log}(N)$	Logarítmica
N	Linear
$N \cdot \text{Log}(N)$	
N^k	Quadrática, cúbica, ...
k^N	Exponencial
$N!$	Fatorial



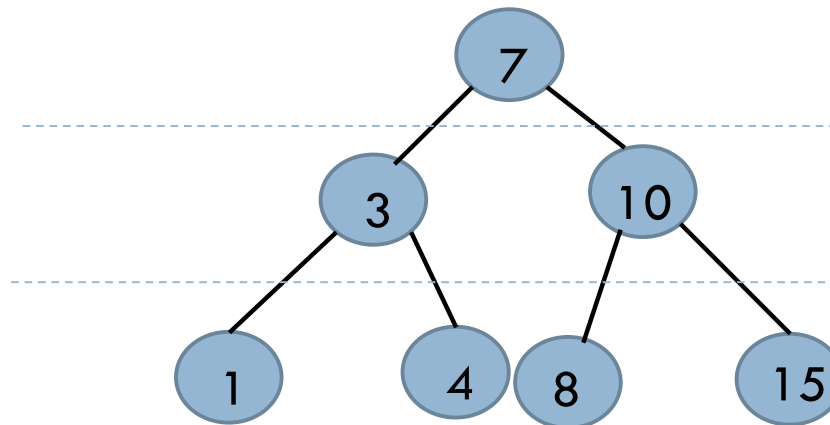
Diminuição da eficiência

Análise de algoritmos

- Funções de custo comuns
- Exemplo $T(n) = 1$
 - ▣ Acesso a uma determinada posição de um array
 - ▣ Acesso indexado
 - ▣ *array*[*i*]

Análise de algoritmos

- Funções de custo comuns
- Exemplo $T(n) = \log(n)$
 - ▣ Busca em um árvore binária de busca (completa)
 - ▣ Custo da busca: altura da árvore
 - $h = \text{floor}(\log(n))$



Número de nós até o nível

$$2^{0+1} - 1 = 1 \text{ nó}$$

$$2^{1+1} - 1 = 3 \text{ nós}$$

$$2^{2+1} - 1 = 7 \text{ nós}$$

Análise de algoritmos

- Funções de custo comuns
- Exemplo $T(n) = n$
 - ▣ Percorrer um array de n elementos

```
for (int i = 0; i < n; i++)  
{  
    ...  
}
```

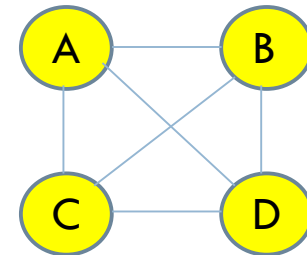

Análise de algoritmos

- Funções de custo comuns

- Exemplo $T(n) = n!$

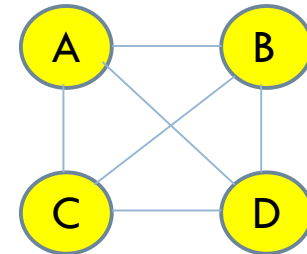
- ▣ Problema do caixeiro viajante

- Suponha que um caixeiro deve visitar n cidades, encerrando a viagem na cidade de origem
- De cada cidade, pode-se ir para todas as outras
- Para se ir de uma cidade para outra existe um custo
 - Tempo
 - Pedágio
 - Custo de gasolina, etc..
 - Ou ainda uma combinação de vários fatores



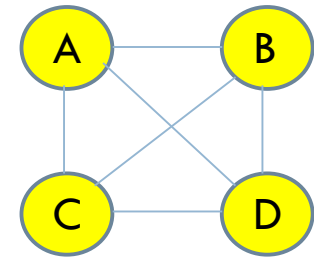
Análise de algoritmos

- Funções de custo comuns
- Exemplo $T(n) = n!$
 - ▣ Problema do caixeiro viajante
 - Sentido da viagem na importa
 - $\text{custo}(a,b) = \text{custo}(b,a)$
 - ▣ Problema: encontrar a melhor rota
 - ▣ Solução força-bruta: calcular todas as rotas possíveis



Análise de algoritmos

- Funções de custo comuns
- Exemplo $T(n) = n!$
 - ▣ Exemplo: suponha 4 cidades (A,B,C e D)
- Possíveis rotas iniciando da cidade A:
 - ▣ ABCDA, ABDCA, ACBDA, ACDBA, ADBCA, ADCBA
- Quantidade total de rotas possíveis: $4 \times 3 \times 2 \times 1 = 4! = 24$
- Para n cidades, $n!$ rotas
 - ▣ $4! = 24$
 - ▣ $10! = 3628800$
 - ▣ $100! \cong 9,3.10^{157}$



Análise de algoritmos

- Notações utilizadas para definir a eficiência de um algoritmo
 - ▣ Notação Θ (Big-Theta)
 - ▣ Notação O (Big-O)
 - ▣ Notação Ω (Big-Omega)
 - ▣ Notação o (Little-o)
 - ▣ Notação ω (Little-omega)

Notação Θ

- Para uma função $g(n)$, denota-se por $\Theta(g(n))$ o conjunto de funções
 - ▣ $\Theta(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c_1, c_2 \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0\}$
 - ▣ Poderíamos dizer que $f(n) \in \Theta(g(n))$
 - Diz-se apenas que $f(n) = \Theta(g(n))$

Notação Θ

□ $f(n) = \Theta(g(n))$

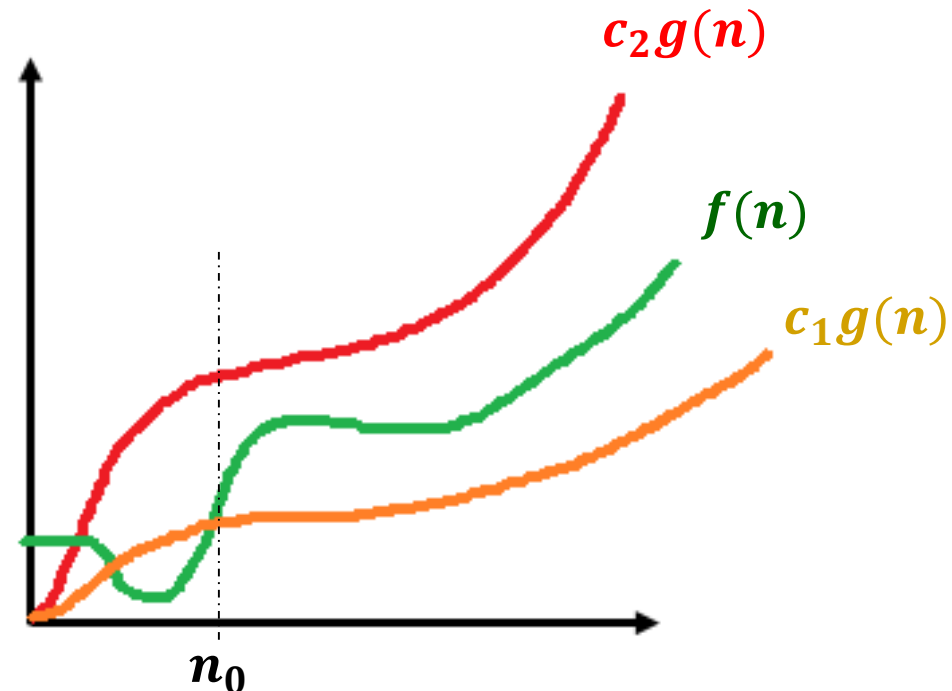
□ Para $n \geq n_0$, $f(n)$ possui complexidade igual a $g(n)$ dentro de um fator constante

□ $g(n)$ é um limite assintoticamente restrito para $f(n)$

□ Exemplos (prova):

1. $2n^2 + 2 = \Theta(n^2)$?

2. $6n^3 \neq \Theta(n^2)$?



Notação Θ

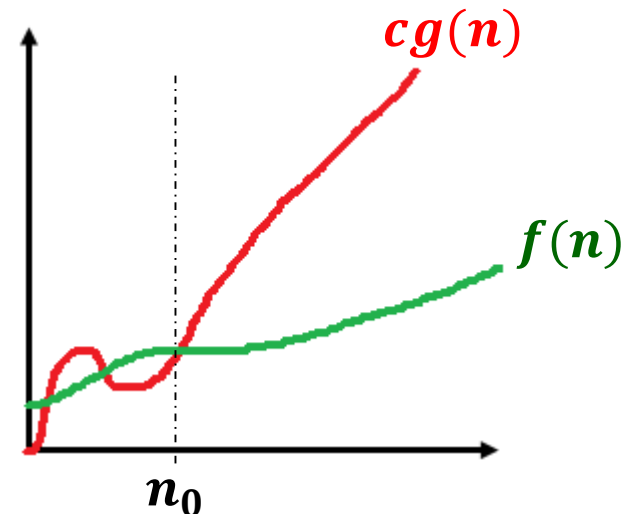
- $f(n) = \Theta(g(n))$
 - ▣ Na determinação de limites assintoticamente restritos:
 - Termos de mais baixa ordem podem ser ignorados
 - Insignificantes para grandes valores de n
 - Em geral:
 - Dado $p(n) = \sum_{i=0}^d a_i n^i$, onde a_i são constantes e $a_d > 0$, temos que $p(n) = \Theta(n^d)$

Notação O

- Para uma função $g(n)$, denota-se por $O(g(n))$ o conjunto de funções
 - ▣ $O(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$
 - ▣ Poderíamos dizer que $f(n) \in O(g(n))$
 - Diz-se apenas que $f(n) = O(g(n))$

Notação O

- $f(n) = O(g(n))$
 - $O(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - $0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$
 - “Algum múltiplo constante de $g(n)$ é um limite assintótico superior sobre $f(n)$ ”



Notação O

□ $f(n) = \Theta(g(n)) \rightarrow f(n) = O(g(n))$

□ Θ é uma notação mais forte que O

□ $\Theta(g(n)) \subseteq O(g(n))$

□ Exemplos:

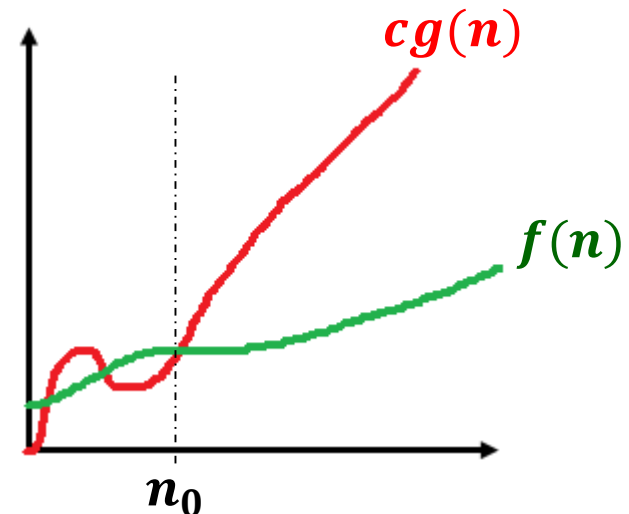
■ $2n^2 + 2 = \Theta(n^2)$

■ Além disso, $2n^2 + 2 = O(n^2)$?

■ Sim!

■ $2n^2 + 2 \neq \Theta(n^3)$

■ Apesar disso, $2n^2 + 2 = O(n^3)$?



Notação O

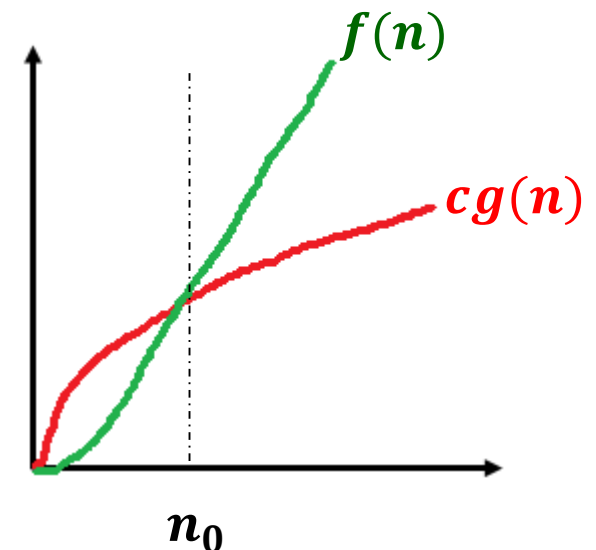
- Utilizando a notação O podemos descrever a complexidade de um algoritmo inspecionando apenas sua estrutura global
 - ▣ Descrevemos, na verdade, um limite superior para o algoritmo
 - ▣ Ex: função `contaTriplas()`
 - ▣ Note que não são executadas, exatamente, n^3 operações
 - Mas, n^3 é um limite superior do número de operações
 - `contaTriplas` é $O(n^3)$
 - ▣ Percorrer um vetor de n elementos é $O(n^2)$?
 - Teoricamente, sim, apesar de esse uso não ser comum
 - Porém, é mais informativo (e bem aceito) afirmar que é $O(n)$!!

Notação Ω

- Para uma função $g(n)$, denota-se por $\Omega(g(n))$ o conjunto de funções
 - ▣ $\Omega(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$
 - ▣ Poderíamos dizer que $f(n) \in \Omega(g(n))$
 - Diz-se apenas que $f(n) = \Omega(g(n))$

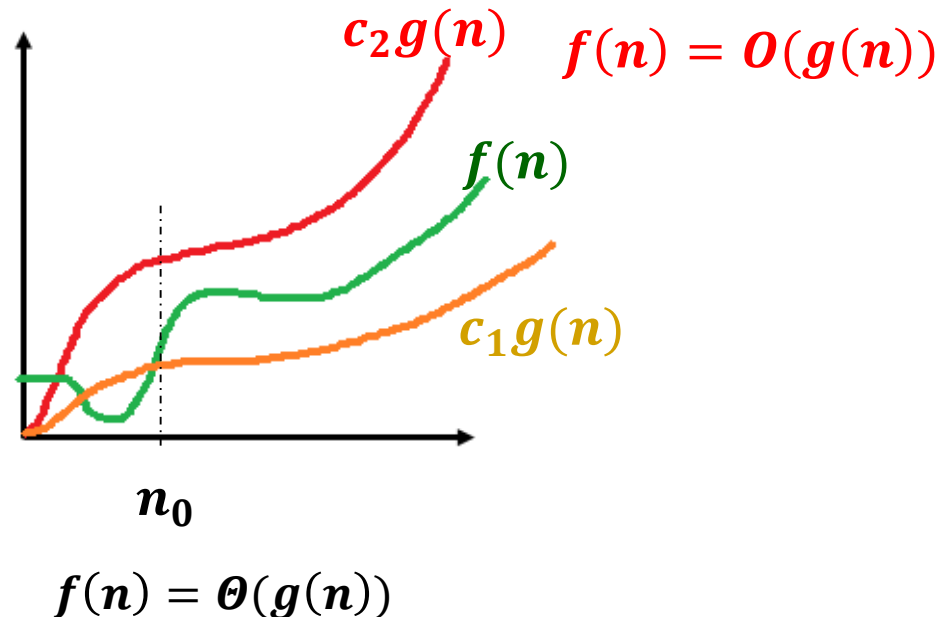
Notação Ω

- $f(n) = \Omega(g(n))$
 - $\Omega(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - $0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$
 - Limite assintótico inferior



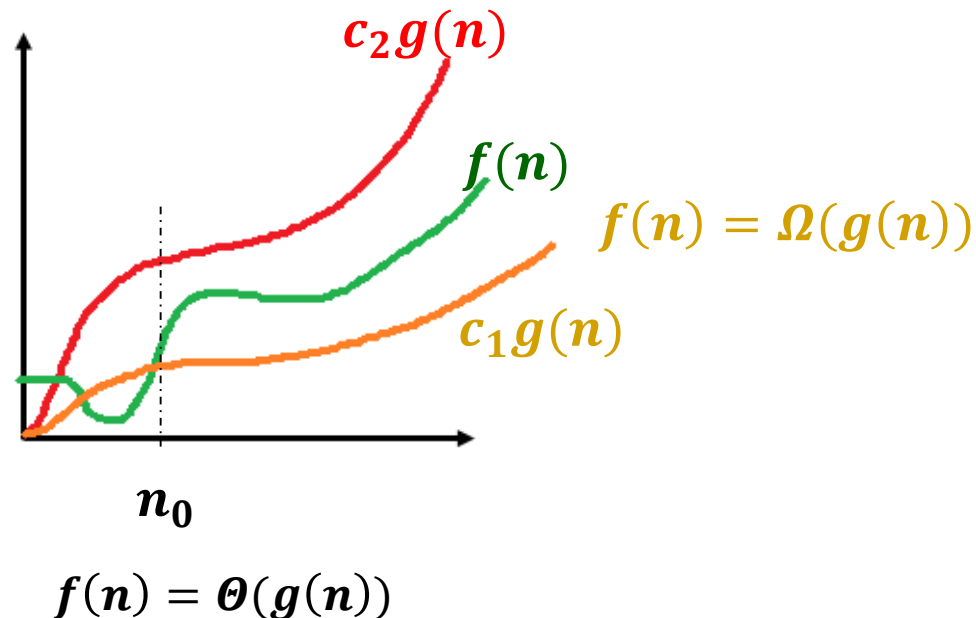
Relação entre Θ e Ω e O

- Para quaisquer funções $f(n)$ e $g(n)$, temos $f(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$



Relação entre Θ e Ω e O

- Para quaisquer funções $f(n)$ e $g(n)$, temos $f(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$



Notação O

- O limite assintótico superior O pode ou não ser assintoticamente restrito
 - ▣ $2n^2 = O(n^2)$ é assintoticamente restrito, mas $n = O(n^2)$, não
- Notação O vs. o
 - ▣ $O(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$
 - ▣ $o(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq f(n) < cg(n) \text{ para todo } n \geq n_0\}$

Notação ω

- O limite assintótico inferior Ω pode ou não ser assintoticamente restrito
 - ▣ $2n^2 = \Omega(n^2)$ é assintoticamente restrito, mas $n^2 = \Omega(n)$, não
- Notação Ω vs. ω
 - ▣ $\Omega(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$
 - ▣ $\omega(g(n)) =$
 $\{f(n): \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que}$
 - ▣ $0 \leq cg(n) < f(n) \text{ para todo } n \geq n_0\}$

Propriedades relacionais

□ Transitividade

- ▣ $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n))$
- ▣ $f(n) = O(g(n))$ e $g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$
- ▣ $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n)) \rightarrow f(n) = \Omega(h(n))$
- ▣ $f(n) = o(g(n))$ e $g(n) = o(h(n)) \rightarrow f(n) = o(h(n))$
- ▣ $f(n) = \omega(g(n))$ e $g(n) = \omega(h(n)) \rightarrow f(n) = \omega(h(n))$

□ Reflexividade

- ▣ $f(n) = \Theta(f(n))$
- ▣ $f(n) = O(f(n))$
- ▣ $f(n) = \Omega(f(n))$

Propriedades relacionais

- Simetria

- ▣ $f(n) = \Theta(g(n))$ sse $g(n) = \Theta(f(n))$

- Simetria de transposição

- ▣ $f(n) = O(g(n))$ sse $g(n) = \Omega(f(n))$

- ▣ $f(n) = o(g(n))$ sse $g(n) = \omega(f(n))$

Comparação assintótica X comparação de números reais

□ Analogia

$f(n) = O(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

Pior, melhor e médio caso

- Ao analisar um algoritmo é possível separá-lo nas seguintes situações:
 - ▣ Pior caso
 - Cenário pessimista
 - Mais utilizado na prática
 - ▣ Melhor caso
 - Menos utilizado
 - Aprova o uso do algoritmo para situações bem específicas
 - ▣ Caso médio
 - Mais difícil de ser analisado
 - Depende da distribuição de probabilidade do conjunto de entrada

Pior, melhor e médio caso

- ❑ Exemplo: busca linear
- ❑ Buscar um determinado elemento em um vetor



Pior, melhor e médio caso

- Exemplo: busca linear
- Buscar um determinado elemento em um vetor



Pior, melhor e médio caso

- ❑ Exemplo: busca linear
- ❑ Buscar um determinado elemento em um vetor



Pior, melhor e médio caso

- ❑ Exemplo: busca linear
- ❑ Buscar um determinado elemento em um vetor



Pior, melhor e médio caso

- Exemplo: busca linear
- Buscar um determinado elemento em um vetor
 - ▣ Caso em que o elemento não está no conjunto deve ser considerado



Pior, melhor e médio caso

- Exemplo: busca linear
- Buscar um determinado elemento em um vetor
 - ▣ Pior caso:
 - Elemento procurado é o último, ou não está no vetor
 - $T(n) = n$
 - $T(n) = O(n)$

Pior, melhor e médio caso

- Exemplo: busca linear
- Buscar um determinado elemento em um vetor
 - ▣ Melhor caso:
 - Elemento procurado é o primeiro
 - $T(n) = 1$
 - $T(n) = O(1)$

Pior, melhor e médio caso

- Exemplo: busca linear
- Buscar um determinado elemento em um vetor
 - ▣ Caso médio: suponha que não há busca sem sucesso
 - ▣ Caso o elemento procurado se encontre na posição i , serão realizadas i verificações
 - ▣ Supondo que cada posição do vetor possua a mesma probabilidade ($p(i) = \frac{1}{n}$) de conter o elemento procurado
 - $T(n) = 1 \frac{1}{n} + 2 \frac{n-1}{n} \frac{1}{n-1} + 3 \frac{n-1}{n} \frac{n-2}{n-1} \frac{1}{n-2} + \dots$
 - $T(n) = (1 + 2 + \dots + (n-1) + n) \cdot \frac{1}{n} = \sum_{i=1}^n i \cdot p(i)$

Pior, melhor e médio caso

- Exemplo: busca linear
- Buscar um determinado elemento em um vetor
 - ▣ Supondo que cada posição do vetor possua a mesma probabilidade ($p(i)$) de conter o elemento procurado
 - $T(n) = (1 + 2 + \dots + (n - 1) + n) \cdot \frac{1}{n}$
 - $2T(n) = (n + (n - 1) + \dots + 2 + 1) \frac{1}{n} + (1 + 2 + \dots + (n - 1) + n) \cdot \frac{1}{n}$
 - $2T(n) = (n(n + 1)) \frac{1}{n} = \frac{n(n+1)}{n}$
 - $T(n) = \frac{n+1}{2}$
 - $T(n) = O(n)$

Leitura recomendada

- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronaldo L.; Stein, Clifford; Algoritmos: teoria e prática
 - ▣ Capítulos 2 e 3.1

